BrandGuard AI

ROUND 3 FINAL

READY FOR JUDGING

Autonomous Multimodal Brand Governance, Verification & Publishing Platform

Live App: https://buildguardai.netlify.app/ **Repository:** emmanuelkeys789/brandguard-ai

Stage: Hackathon Round 3 of 3 **Date:** October 2026

1. Executive Summary & Purpose

BrandGuard AI is an enterprise brand governance and multimodal compliance system built to prevent brand erosion, legal risk, inaccessible creative distribution, and messaging inconsistencies before publication.

Modern content workflows face an acute challenge: generic LLMs frequently hallucinate brand rules, generate unverified claims, or fail to catch subtle visual and accessibility violations. BrandGuard AI solves this with a **hybrid architecture**:

- **Deterministic Rule Verification:** Mathematical precision for brand guidelines, trademark symbols, prohibited claims, and legal disclaimers. 100% reproducible and zero hallucination.
- **Advisory Contextual AI Reasoning:** High-level voice and tone critique powered by real LLM vision and reasoning models (OpenAI `gpt-4o-mini` & Google Gemini `gemini-1.5-flash`), offering substantive drop-in rewrites without altering deterministic compliance scores.
- **Multimodal Creative & Alt-Text Audit:** Direct ingestion and analysis of marketing imagery, clear-space detection, WCAG contrast checks, and author-supplied alt-text verification.
- **Role-Based Access Control (RBAC):** Strict separation between Editors (drafting and checks) and Reviewers/Admins (approvals, rejections, change requests, and publishing).
- **Signed Idempotent Webhook Dispatch:** Automated downstream distribution with HMAC-SHA256 signatures, SSRF destination protection, and offline demo simulation.

2. Status of Exact Round 2 Commitments

In the Round 2 submission, three specific features were committed for delivery in Round 3. All three commitments are **100% completed, fully tested, and judge-ready**:

Round 2 Commitment	Delivered Scope in Round 3	Status	Verification Evidence
1. Multi-Modal Visual Asset Inspection	• Ingestion of PNG, JPEG, WEBP creatives up to 4 MB. • In-memory canvas creative generator (no external dependencies). • Rigorous author alt-text accessibility auditing (detects	COMPLETED	• `multimodalReview.test.ts` (22 tests passing). • Live endpoint verified: `POST /api/multimodal-review` returned HTTP 200 with OpenAI vision findings.

Round 2 Commitment	Delivered Scope in Round 3	Status	Verification Evidence
	missing and generic alt-text). • Observability evidence callouts vs. uncertain visual judgments. • Dual provider support: Google Gemini Vision & OpenAI Vision with fallback.		
2. Enterprise RBAC & Role Governance	• Three distinct roles: **Editor**, **Reviewer**, and **Admin**. • Editors strictly prevented from approving, rejecting, or publishing content. • Reviewers and Admins hold signoff and webhook publishing authority. • Server-side token authorization via Supabase Auth and `user_roles` table. • Transparent simulated demo personas for seamless judging.	COMPLETED	• `governanceWorkflow.test.ts` (26 tests passing). • Permission matrix enforced in UI buttons, context handlers, and storage layer.
3. Direct Publishing Webhooks & CMS Integrations	• Serverless endpoint (`POST /api/publish`) with Netlify Functions v2. • Publishing strictly gated behind explicit human approval (`Approved`). • HMAC-SHA256 signature calculation with server secret protection. • SSRF protection rejecting loopback, cloud metadata, and private IP ranges. • Deterministic idempotency key derivation & HTTP 409 repeated publish guards. • Offline demo simulation with honest `SIMULATED DEMO DELIVERY` labeling.	COMPLETED	• `publishingHandler.test.ts` (28 tests passing). • `publishingService.test.ts` (5 tests passing). • Interactive payload & signature inspection drawer in UI.

3. Complete Quality & Verification Results

The codebase passes all automated regression, linting, type-checking, and build validation suites with zero warnings and zero errors:

Verification Check	Command Executed	Result	Details
Automated Test Suite	`npm test`	**197 / 197 PASS**	50 test suites, 0 failures, 0 skipped, completed in 1.12s.
Linter Static Analysis	`npm run lint`	**0 ERRORS / 0 WARNINGS**	Oxlint scanned 73 files with 116 rules in 173ms.
TypeScript Compiler	`npx tsc -b --noEmit`	**CLEAN (CODE 0)**	Zero type errors across all client, service, and server files.
Production Build Rollup	`npm run build`	**BUILT IN 364MS**	Vite v8.3.5 generated clean production bundle without secret leaks.

4. Three-Tier Verification Breakdown

To ensure honesty for hackathon evaluation, BrandGuard AI explicitly distinguishes what is verified automatically, in local demo mode, and in live cloud deployment:

Tier	Target Environment	Capabilities Verified	Verification Method
Tier 1: Automated Quality	Node.js Test Runner / CI	• State transition legality and duplicate prevention. • SSRF rejection on private IPs and cloud metadata. • HMAC signature math and idempotency key derivation. • Prompt isolation and delimiter escaping. • Schema parity between migrations and data mappers.	197 automated tests passing in `src/services/*.test.ts`.
Tier 2: Local Demo Experience	Local Dev Server (`localhost:5173`)	• 1-click **"Draft + Creative Logo"** asset generation. • Tri-stream findings inspection (Policy, AI Tone, Visual Compliance). • 1-click **"Apply to Draft"** text remediation. • Simulated persona	Interactive browser dev server verification.

Tier	Target Environment	Capabilities Verified	Verification Method
		switcher (Admin, Reviewer, Editor). • Gated publishing notice and offline demo publishing simulation. • LocalStorage rehydration preserving review history across refresh.	
Tier 3: Live Cloud Production	Netlify Edge (`buildguardai.netlify.app`)	• SPA static asset delivery and client routing via `public/_redirects`. • Production endpoint: `POST /api/ai-tone-review` (OpenAI `gpt-4o-mini`). • Production endpoint: `POST /api/multimodal-review` (OpenAI Vision).	Live HTTP/2 cURL requests returning genuine structured findings.

5. Security & Architectural Invariants

> **Zero Secret Leakage Invariant:** Server API keys (`OPENAI_API_KEY`, `GEMINI_API_KEY`, `SUPABASE_SERVICE_ROLE_KEY`, `PUBLISHING_WEBHOOK_SECRET`) are strictly isolated to backend functions and never bundled into frontend assets. Production bundle inspection of `dist/` confirmed zero secret key occurrences.

- **Untrusted Content Isolation:** Draft copy is encapsulated within `<UNTRUSTED_DRAFT_CONTENT>` tags. System prompts strictly instruct models to ignore instructions embedded in marketing drafts.
- **SSRF Defense:** Webhook destination validation rejects private IP ranges (`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`), link-local IPs (`169.254.0.0/16`), loopback (`127.0.0.1`), cloud metadata hosts, and requires HTTPS in production.
- **Terminal State Idempotency:** Once a review reaches `Published` status, subsequent publish attempts return an immediate HTTP 409 Conflict to protect downstream systems from duplicate dispatches.

6. Honest Transparency Disclosures & Limitations

> **Transparency for Judges:**
>
> - **Simulated Demo Personas:** The UI persona switcher simulates Admin, Reviewer, and Editor roles client-side for ease of evaluation. Real multi-tenant authorization requires authentic Supabase Auth credentials backed by the server `user_roles` table.
> - **LocalStorage vs. Cloud Ledger:** In local demo mode, records are stored in browser `LocalStorage`. While lifecycle rules are enforced in memory, browser storage is not cryptographically immutable. Authoritative immutability requires Cloud PostgreSQL with Row-Level Security.
> - **Live Webhook Endpoint Configuration:** Real webhook dispatch via `/api/publish` requires server environment configuration (`PUBLISHING_WEBHOOK_URL` , `PUBLISHING_WEBHOOK_SECRET` , `SUPABASE_SERVICE_ROLE_KEY`). In demo mode, offline simulation provides complete workflow verification without external CMS webhooks.

7. Quick 2-Minute Judge Evaluation Script

1. **Open Application:** Navigate to the **Check Content** tab.
2. **Load Sample with Creative:** Click the blue **"Draft + Creative Logo"** button. Notice that both the marketing text (with deliberate superlatives) and the in-memory BrandGuard creative banner with accessibility alt-text populate immediately.
3. **Run Content Check:** Click **"Run Content Check"**.
4. **Inspect Tri-Stream Findings:**
   - *Policy Violations:* Deterministic engine flags unsupported claims (*"undisputed #1"*, *"100% immunity"*).
   - *AI Tone Suggestions:* AI critique evaluates voice and recommends substantive drop-in replacements.
   - *Visual Compliance:* Inspect visual findings and author alt-text evaluation.
5. **Apply Remedy:** Click **"Apply to Draft"** on an AI suggestion or rule correction; observe the draft textarea update instantly.
6. **Submit Review:** Click **"Save Review to Past Reviews"**.
7. **Inspect Gated Governance:** Open **Past Reviews**, click the submitted review, and observe the **"Publishing Gated"** alert.
8. **Approve & Simulate Webhook:** Switch to **Reviewer** or **Admin** persona, click **"Approve"**, open **"Inspect Webhook Payload & Signature"**, and click **"Simulate Webhook Delivery (Offline Demo)"**. Observe the `SIMULATED DEMO DELIVERY` badge and verify repeated publishing is blocked.

8. Repository & Deployment Metadata

- **Repository URL:** https://github.com/emmanuelkeys789/brandguard-ai.git
- **Live Application URL:** https://buildguardai.netlify.app/
- **Deployment Configuration:** Netlify Functions v2 (`netlify.toml`) + Vite build rollup.

- **Database Migrations:** 4 sequential versioned SQL migrations documented in `supabase/migrations/` with unified schema in `supabase/schema.sql`.